

LeetCode Deep Dive

Binary Search on the Answer: Turning Optimization into a Bound
Week 14 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

The same bounding trick, in a new costume

- This week's lecture unifying lesson was: never compute OPT directly – squeeze the algorithm against a quantity you *can* reason about.
- Both problems below are “minimize the worst-case value V ” problems where **feasibility at value V is monotonic**: if V works, every larger value also works. That single fact turns an optimization problem into a fast decision problem, and **binary search** does the rest – no LP, no matching, no MST needed.
- We pick one **Medium** (*Koko Eating Bananas*) and one **Hard** (*Split Array Largest Sum*), both official Week 14 practice problems, and apply the same four-step process used in Week 3: naive instinct → why it hurts → the monotonicity insight → optimal algorithm and implementation tricks.

Problem statement

LeetCode 875 – Koko Eating Bananas (Medium)

There are n piles of bananas; pile i has `piles[i]` bananas. Koko eats at a constant integer speed of k bananas/hour. Each hour she picks one pile and eats $\min(k, \text{pile size})$ bananas from it (a pile smaller than k finishes in that hour without carrying leftover speed to another pile). Find the **minimum** integer k so she can finish all piles within h hours.

Constraints (LeetCode): $1 \leq n \leq 10^4$, $1 \leq \text{piles}[i] \leq 10^9$, $n \leq h \leq 10^9$.

Worked example

piles = [3, 6, 7, 11], h = 8

speed k	pile 3	pile 6	pile 7	pile 11	total hours	fits in $h=8$?
3	1	2	3	4	10	no
4	1	2	2	3	8	yes

Hours for a pile at speed k is $\lceil \text{pile}/k \rceil$. **Answer:** 4 – the smallest speed for which the total is $\leq h$.

First instinct: scan every speed from 1 upward

```
1: for  $k \leftarrow 1, 2, 3, \dots, \max(\text{piles})$  do  
2:   if HOURSNEEDED(piles,  $k$ )  $\leq h$  then  
3:     return  $k$   
4:   end if  
5: end for
```

Why this is bad

Each check costs $O(n)$. In the worst case you scan up to $\max(\text{piles})$ speeds. Plug in real constraints: $\text{piles}[i]$ up to 10^9 , n up to $10^4 \Rightarrow$ up to $10^9 \times 10^4 = 10^{13}$ operations. This will never finish.

The structural insight

- Define $\text{feasible}(k) = [\text{Koko finishes within } h \text{ hours at speed } k]$. Increasing k can only *decrease or keep* the hours needed for each pile ($\lceil p/k \rceil$ is non-increasing in k) – so feasible is **monotonic**: false, false, ..., false, true, true, ..., true.
- Finding the boundary of a monotonic boolean sequence is exactly what **binary search** does, in $O(\log(\max(\text{piles})))$ probes instead of $O(\max(\text{piles}))$ linear steps.
- This is the same shape as this week's bounding technique: we never evaluate every candidate k – we bound the search range and halve it every step.

Optimal algorithm: binary search on the answer

```
1: function HOURSNEEDED(piles,  $k$ )
2:   return  $\sum_{p \in \text{piles}} \lceil p/k \rceil$ 
3: end function
4: function MINEATINGSPEED(piles,  $h$ )
5:    $lo \leftarrow 1$ ;  $hi \leftarrow \max(\text{piles})$ 
6:   while  $lo < hi$  do
7:      $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
8:     if HOURSNEEDED(piles,  $mid$ )  $\leq h$  then
9:        $hi \leftarrow mid$ 
10:    else
11:       $lo \leftarrow mid + 1$ 
12:    end if
13:  end while
14:  return  $lo$ 
15: end function
```

Cost: $O(n \log(\max(\text{piles})))$.

▷ mid works: the answer is $\leq mid$

▷ mid fails: the answer is $> mid$

Trace on the worked example

piles = [3, 6, 7, 11], h = 8 – binary search for the feasibility boundary:

step	<i>lo</i>	<i>hi</i>	<i>mid</i>	hours at <i>mid</i> / action
1	1	11	6	$1+1+2+2=6 \leq 8$: $hi \leftarrow 6$
2	1	6	3	$1+2+3+4=10 > 8$: $lo \leftarrow 4$
3	4	6	5	$1+2+2+3=8 \leq 8$: $hi \leftarrow 5$
4	4	5	4	$1+2+2+3=8 \leq 8$: $hi \leftarrow 4$

$lo = hi = 4$: loop ends. **Answer:** 4 – matches the worked example.

Complexity: naive vs. optimal

Approach	Time	At $\max(\text{piles}) = 10^9, n = 10^4$
Linear scan over k	$O(n \cdot \max(\text{piles}))$	$\sim 10^{13}$ ops
Binary search on k	$O(n \log(\max(\text{piles})))$	$\sim 3 \times 10^5$ ops

Python implementation

```
1 def minEatingSpeed(piles, h):
2     def hours_needed(k):
3         return sum(-(-p // k) for p in piles)    # ceil(p / k) without floats
4
5     lo, hi = 1, max(piles)
6     while lo < hi:
7         mid = (lo + hi) // 2
8         if hours_needed(mid) <= h:
9             hi = mid
10        else:
11            lo = mid + 1
12    return lo
```

Implementation tips & tricks (the programming side)

- **Ceiling division without floats.** $-(-p // k)$ (or $(p + k - 1) // k$) computes $\lceil p/k \rceil$ using only integer arithmetic. `math.ceil(p / k)` looks equivalent but risks a wrong answer: Python floats have 53 bits of mantissa, and p up to 10^9 divided by a small k can round incorrectly at the boundary in rare cases – pure integer arithmetic sidesteps this entirely.
- **The `hi = mid` vs. `hi = mid - 1` choice is not cosmetic.** We are searching for the *leftmost* k where `feasible(k)` is true, and *mid* itself might already be that answer – so on success we must keep *mid* in play (`hi = mid`), not discard it (`hi = mid - 1`, the pattern used when searching for a value *equal* to a target). Mixing up the two binary-search templates is the single most common bug in this problem.
- `hi = max(piles)` is a *provably sufficient* upper bound (eating at that speed finishes every pile in exactly one hour each) – no need to guess a larger bound.

Problem statement

LeetCode 410 – Split Array Largest Sum (**Hard**)

Given an integer array `nums` and an integer m , split `nums` into exactly m non-empty **contiguous** subarrays so as to **minimize the largest sum** among the m subarrays. Return that minimized largest sum.

Constraints (LeetCode): $1 \leq n \leq 1000$, $0 \leq \text{nums}[i] \leq 10^6$, $1 \leq m \leq \min(50, n)$.

Worked example

`nums = [7, 2, 5, 10, 8]`, `m = 2` – every way to cut it into 2 contiguous parts:

split	part sums (largest)
<code>[7] [2, 5, 10, 8]</code>	7, 25 (largest 25)
<code>[7, 2] [5, 10, 8]</code>	9, 23 (largest 23)
<code>[7, 2, 5] [10, 8]</code>	14, 18 (largest 18)
<code>[7, 2, 5, 10] [8]</code>	24, 8 (largest 24)

Answer: 18 – the split `[7, 2, 5] | [10, 8]` minimizes the largest part sum.

First instinct: try every split, or DP over split points

- **Pure brute force:** enumerate all $\binom{n-1}{m-1}$ ways to choose $m-1$ cut points among $n-1$ gaps, score each, keep the best. For $n=1000$, $m=50$ this is astronomically large – not even close to feasible.
- **A better naive instinct – DP:** let $f(i, j)$ be the best (minimized largest-part) answer for splitting the first i elements into j parts:

$$f(i, j) = \min_{k < i} \max(f(k, j-1), \text{sum}(k+1..i)).$$

This is a legitimate polynomial algorithm, $O(n^2m)$ with prefix sums. At $n=1000$, $m=50$: $1000^2 \times 50 = 5 \times 10^7$ – survivable, but far more work than necessary, and it is a genuinely different (harder to get right) algorithm than the one below.

The structural insight

- Define $\text{feasible}(L) = [\text{nums can be split into } \leq m \text{ contiguous parts, each with sum } \leq L]$. Larger L can only make splitting *easier* (fewer parts needed) – feasible is **monotonic** in L , exactly like Koko's speed.
- Checking $\text{feasible}(L)$ needs no DP at all: greedily pack elements into the current part until the *next* element would push the sum over L , then start a new part. Count the parts used; feasible iff that count $\leq m$. This greedy check is $O(n)$.
- So: binary search over $L \in [\max(\text{nums}), \text{sum}(\text{nums})]$ (a single part can never be smaller than the largest element, and can never need to exceed the sum of everything), each probe an $O(n)$ greedy check.

Optimal algorithm: the $O(n)$ feasibility check

```
1: function FEASIBLE(nums,  $m$ ,  $L$ )
2:    $parts \leftarrow 1$ ;  $cur \leftarrow 0$ 
3:   for  $x$  in nums do
4:     if  $cur + x > L$  then
5:        $parts \leftarrow parts + 1$ ;  $cur \leftarrow x$ 
6:     else
7:        $cur \leftarrow cur + x$ 
8:     end if
9:   end for
10:  return  $parts \leq m$ 
11: end function
```

▷ start a new part

Greedy pack elements into the current part until the next one would overflow it, then start a new part – count the parts used and compare to m .

Optimal algorithm: binary search on the answer

```
1: function SPLITLARGESTSUM(nums,  $m$ )
2:    $lo \leftarrow \max(\text{nums}); hi \leftarrow \text{sum}(\text{nums})$ 
3:   while  $lo < hi$  do
4:      $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
5:     if FEASIBLE(nums,  $m$ ,  $mid$ ) then  $hi \leftarrow mid$ 
6:     else  $lo \leftarrow mid + 1$ 
7:     end if
8:   end while
9:   return  $lo$ 
10: end function
```

Cost: $O(n \log(\text{sum}(\text{nums})))$ – same shape as Koko's search, with a greedy-packing feasibility check in place of a sum-of-ceilings check.

Trace on the worked example

$\text{nums} = [7, 2, 5, 10, 8]$, $m = 2$ – $lo = \max = 10$, $hi = \text{sum} = 32$.

step	lo	hi	mid	parts needed at limit mid / action
1	10	32	21	$[7, 2, 5], [10, 8]$: 2 parts ≤ 2 : $hi \leftarrow 21$
2	10	21	15	$[7, 2, 5], [10], [8]$: 3 parts > 2 : $lo \leftarrow 16$
3	16	21	18	$[7, 2, 5], [10, 8]$: 2 parts ≤ 2 : $hi \leftarrow 18$
4	16	18	17	$[7, 2, 5], [10], [8]$: 3 parts > 2 : $lo \leftarrow 18$

$lo = hi = 18$: loop ends. **Answer:** 18 – matches the worked example.

Complexity: naive vs. optimal

Approach	Time	At $n = 1000, m = 50$
Brute-force enumerate cut points	$O\left(\binom{n-1}{m-1}\right)$	astronomical
DP over split points	$O(n^2m)$	$\sim 5 \times 10^7$ ops
Binary search + greedy check	$O(n \log(\text{sum}))$	$\sim 3 \times 10^4$ ops

Binary search is not just faster here – it is also a *simpler* algorithm to implement correctly than the $O(n^2m)$ DP.

Python implementation

```
1 def splitArray(nums, m):
2     def feasible(limit):
3         parts, cur = 1, 0
4         for x in nums:
5             if cur + x > limit:
6                 parts += 1
7                 cur = x
8                 if parts > m:           # early exit: already infeasible
9                     return False
10            else:
11                cur += x
12        return True
13
14    lo, hi = max(nums), sum(nums)
15    while lo < hi:
16        mid = (lo + hi) // 2
17        if feasible(mid):
18            hi = mid
19        else:
20            lo = mid + 1
21    return lo
```

Implementation tips & tricks (the programming side)

- **Early exit inside the greedy check.** The moment $\text{parts} > m$, the rest of the array cannot rescue feasibility – returning immediately (line 9 above) avoids scanning the remaining elements. On adversarial inputs (e.g. every element forces a new part) this turns a wasted full pass into an early bailout.
- **Check $\text{parts} > m$ after incrementing, not before.** A common off-by-one bug is comparing parts to m before the increment that just happened – that silently allows one extra part.
- $\text{lo} = \max(\text{nums})$, not $\text{lo} = 0$ or $\text{lo} = 1$. A limit smaller than the largest single element can *never* be feasible (that element alone would overflow its part), so starting the search range there is not just an optimization – it rules out probing values that would waste a whole $O(n)$ feasibility check only to learn something already known for free.

Summary: one trick, two problems

- 1 Both problems ask “minimize the worst-case value of something,” and both have a **monotonic feasibility test**: once a candidate value works, every larger value also works – check for this before reaching for the technique.
- 2 The pattern is always the same: (a) find a provably sufficient $[lo, hi]$ range, (b) write an $O(n)$ -ish `feasible(mid)` check (often a small greedy algorithm, as in Split Array), (c) binary-search using the “*does mid work?*” template (`hi = mid` on success), never the “*equals the target?*” template.
- 3 Same unifying idea as this week’s approximation proofs: avoid computing OPT directly, bound it and squeeze – here the “bound” is a candidate answer, and binary search is the squeeze.
- 4 The DP alternative for Split Array is a reminder that a *correct, polynomial* algorithm is not automatically the *simplest* one; monotonic structure can replace a whole DP table with a dozen lines of binary search.

Keep practicing

LEETCODE.md lists more Week 14 problems that reuse this exact technique (*Minimize Max Distance to Gas Station*, *Minimize the Maximum Difference of Pairs*) plus others that need different tools (*Maximum Performance of a Team*, *Smallest Sufficient Team*, *Minimum Number of Arrows to Burst Balloons*) – try spotting which category each one falls into before you start coding.